

Documento de Trechos de Código - BarberMove (RPC)

Data: 2026-02-15

Titular/Autor: Allan de Jesus Pereira Siqueira

Versão do Software: 1.0.0

Resumo

Este documento reúne trechos reais e representativos do software BarberMove para fins de depósito no RPC (INPI

). Os trechos estão organizados por módulo e mostram configuração da API, modelos, regras de negócio e fluxo de interface.

Sumário

1. Contexto do Software
2. Backend - API e Modelos
3. Frontend - Interface e Fluxos
4. Observações

1. Contexto do Software

- Nome do software: BarberMove
- Finalidade: plataforma para agendamento, gestão de barbeiros, barbearias e clientes
- Linguagens principais: Python (FastAPI), JavaScript/React

2. Backend - API e Modelos

2.1 app/main.py (configuração e rotas)

```
from fastapi import FastAPI, HTTPException, WebSocket, WebSocketDisconnect
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
from dotenv import load_dotenv
import os
```

```
load_dotenv()
```

```
from .routes import router
from .routes_extras import router as router_extras
from .routes_documentos import router as router_documentos
from .routes_disponibilidade import router as router_disponibilidade
from .routes_relatorio import router as router_relatorio
from .routes_servicos import router as router_servicos
from .routes_pagamentos import router as router_pagamentos
from .routes_freelancer import router as router_freelancer
from .routes_cadeiras import router as router_cadeiras
from .routes_avaliacoes import router as router_avaliacoes
from .routes_barbearias import router as router_barbearias
from .routes_notificacoes import router as router_notificacoes
from .routes_precos import router as router_precos
from .routes_analytics import router as router_analytics
from .admin_routes import router as router_admin
```

```

from .routes_legais import router as router_legais
from .routes_fixes import router as router_fixes
from .routes_aprovacoes import router as router_aprovacoes
from .database import Base, engine, init_db
from starlette.staticfiles import StaticFiles
from fastapi.responses import FileResponse
import pathlib

@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()
    yield

app = FastAPI(title="BarberMove API", lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(router, prefix="/api/v1")
app.include_router(router_extras, prefix="/api/v1")
app.include_router(router_documentos, prefix="/api/v1/documentos")
app.include_router(router_disponibilidade, prefix="/api/v1")
app.include_router(router_relatorio, prefix="/api/v1")
app.include_router(router_servicos, prefix="/api/v1")
app.include_router(router_pagamentos, prefix="/api/v1")
app.include_router(router_freelancer)
app.include_router(router_cadeiras)
app.include_router(router_avaliacoes)
app.include_router(router_barbearias)
app.include_router(router_notificacoes)
app.include_router(router_precos)
app.include_router(router_analytics)
app.include_router(router_admin)
app.include_router(router_fixes, prefix="/api/v1")
app.include_router(router_aprovacoes, prefix="/api/v1")
app.include_router(router_legais, prefix="/api/v1")

```

2.2 app/models.py (modelos e entidades)

```

class StatusAgendamento(str, enum.Enum):
    PENDENTE = "pendente"
    CONFIRMADO = "confirmado"
    CONCLUIDO = "concluido"
    CANCELADO = "cancelado"

class Usuario(Base):
    __tablename__ = "usuarios"
    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, index=True)
    nome = Column(String)
    senha_hash = Column(String)
    tipo = Column(String) # cliente, barbeiro, barbearia
    endereco = Column(String, nullable=True)
    telefone = Column(String, nullable=True)
    cpf = Column(String, nullable=True, unique=True)

```

```

cnpj = Column(String, nullable=True, unique=True)
foto_perfil = Column(String, nullable=True)
perfil_aprovado = Column(Boolean, default=False)
disponivel = Column(Boolean, default=False)
criado_em = Column(DateTime, default=datetime.utcnow)

class Barbearia(Base):
    __tablename__ = "barbearias"
    id = Column(Integer, primary_key=True, index=True)
    usuario_id = Column(Integer, ForeignKey("usuarios.id"))
    nome = Column(String)
    endereco = Column(String)
    telefone = Column(String, nullable=True)
    cadeira_livre = Column(Boolean, default=True)

class Servico(Base):
    __tablename__ = "servicos"
    id = Column(Integer, primary_key=True, index=True)
    barbearia_id = Column(Integer, ForeignKey("barbearias.id"), nullable=False, index=True)
    nome = Column(String, nullable=False)
    descricao = Column(String, nullable=True)
    categoria = Column(String, nullable=False, index=True)
    valor = Column(Float, nullable=False)
    duracao_minutos = Column(Integer, default=30)
    ativo = Column(Boolean, default=True)

class Chamado(Base):
    __tablename__ = "chamados"
    id = Column(Integer, primary_key=True, index=True)
    cliente_id = Column(Integer, ForeignKey("usuarios.id"))
    barbeiro_id = Column(Integer, ForeignKey("usuarios.id"), nullable=True)
    servico_id = Column(Integer, ForeignKey("servicos.id"))
    barbearia_id = Column(Integer, ForeignKey("barbearias.id"))
    cadeira_id = Column(Integer, ForeignKey("cadeiras.id"), nullable=True)
    data_hora_inicio = Column(DateTime, nullable=True, index=True)
    data_hora_fim = Column(DateTime, nullable=True)
    status = Column(String, default=StatusAgendamento.PENDENTE)

```

2.3 app/routes.py (regras de negocio e validacoes)

```

def calcular_split_pagamento(valor_total: float) -> dict:
    comissao_plataforma = round(valor_total * 0.15, 2)
    valor_freelancer = round(valor_total * 0.45, 2)
    valor_dono = round(valor_total - comissao_plataforma - valor_freelancer, 2)
    return {
        'valor_total': valor_total,
        'comissao_plataforma': comissao_plataforma,
        'valor_freelancer': valor_freelancer,
        'valor_dono': valor_dono
    }

def is_horario_disponivel(db: Session, barbeiro_id: int, inicio: datetime, fim: datetime) -> bool:
    from sqlalchemy import and_
    conflito = db.query(models.Chamado).filter(
        and_(
            models.Chamado.barbeiro_id == barbeiro_id,
            models.Chamado.status != models.StatusAgendamento.CANCELADO.value,
            (
                (models.Chamado.data_hora_inicio.is_(None)) |
                (models.Chamado.data_hora_fim.is_(None)) |

```

```

        and_(
            models.Chamado.data_hora_inicio < fim,
            models.Chamado.data_hora_fim > inicio
        )
    )
).first()
if conflito:
    return False
return True

```

2.4 app/routes_cadeiras.py (controle de cadeira e presenca)

```

@router.post("/presenca", response_model=dict)
def marcar_presenca_freelancer(
    payload: CadeiraPresencaRequest,
    db: Session = Depends(get_db),
    usuario_atual = Depends(get_current_user)
):
    barbearia = db.query(Barbearia).filter(
        Barbearia.usuario_id == usuario_atual.id
    ).first()
    if not barbearia:
        raise HTTPException(status_code=404, detail="Barbearia nao encontrada")

    cadeira = db.query(Cadeira).filter(
        Cadeira.id == payload.cadeira_id,
        Cadeira.barbearia_id == barbearia.id
    ).first()
    if not cadeira:
        raise HTTPException(status_code=404, detail="Cadeira nao encontrada")

    freelancer = db.query(Usuario).filter(Usuario.id == payload.freelancer_id).first()
    if not freelancer or freelancer.tipo != "barbeiro":
        raise HTTPException(status_code=404, detail="Freelancer nao encontrado")

    if cadeira.status == StatusCadeira.OCUPADA and cadeira.freelancer_id != payload.freelancer_id:
        nome = cadeira.freelancer.nome if cadeira.freelancer else "Freelancer"
        raise HTTPException(status_code=400, detail=f"BRB ocupada por {nome}")

    cadeira.status = StatusCadeira.OCUPADA
    cadeira.freelancer_id = payload.freelancer_id
    cadeira.ocupada_em = datetime.now()
    cadeira.chamado_id = None

    freelancer.disponivel = True
    db.commit()
    db.refresh(cadeira)

```

2.5 app/routes_freelancer.py (busca por proximidade)

```

@router.get("/proximos", response_model=List[FreelancerResponse])
def buscar_freelancers_proximos(
    latitude: float,
    longitude: float,
    raio_km: float = 5.0,
    especialidade: str = None,
    db: Session = Depends(get_db)
):
    query = db.query(

```

```

        Freelancer,
        Usuario.nome,
        Usuario.foto_perfil,
        Usuario.latitude,
        Usuario.longitude
    ).join(Usuario, Freelancer.usuario_id == Usuario.id).filter(
        Freelancer.status_pausado == False,
        Usuario.latitude.isnot(None),
        Usuario.longitude.isnot(None)
    )
    if especialidade:
        query = query.join(EspecialidadeFreelancer).filter(
            EspecialidadeFreelancer.tipo == especialidade
        )

```

3. Frontend - Interface e Fluxos

3.1 barbermove/src/App.jsx (estado principal e UI)

```

import React, { useState, useEffect } from 'react';
import {
    User, Scissors, Store, MapPin,
    Logout, CheckCircle, AlertCircle, ArrowRight,
    History, Search, X, Star, Navigation, Bell, CreditCard, Lock, Calendar
} from 'lucide-react';
import PoliticaPrivacidade from './components/PoliticaPrivacidade';
import TermosDeUso from './components/TermosDeUso';
import TelaPerfilUsuario from './components/TelaPerfilUsuario';
import TelaPagamento from './components/TelaPagamento';
import RatingComponent from './components/RatingComponent';
import AbaPadronizadaAvaliacoes from './components/AbaPadronizadaAvaliacoes';
import PaymentSection from './components/PaymentSection';
import ProfileCard from './components/ProfileCard';
import ClientDashboard from './components/ClientDashboard';
import BarberDashboard from './components/BarberDashboard';
import ShopDashboard from './components/ShopDashboard';
import { useLiveJobs, useRealTimeUpdates } from './hooks/useRealTimeUpdates';

const API_URL = import.meta.env.VITE_API_URL || "http://localhost:8000";
const WS_URL = import.meta.env.VITE_WS_URL || "ws://localhost:8000/ws/notificacoes";
const BRAND_LOGO = "/brand-logo.png";

```

3.2 barbermove/src/components/TelaPerfilUsuario.jsx (upload e perfil)

```

const handleSaveProfile = async () => {
    setLoading(true);
    try {
        let fotoPerfil = previewImage;

        if (previewImage && previewImage.startsWith('data:')) {
            const response = await fetch(previewImage);
            const blob = await response.blob();

            const formData = new FormData();
            formData.append('file', blob, 'foto_perfil.jpg');
            formData.append('pasta', 'perfil');

```

```

const uploadRes = await fetch(`${API_URL}/api/v1/upload/imagem`, {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${token}`
  },
  body: formData
});

if (!uploadRes.ok) {
  const error = await uploadRes.json();
  throw new Error('Erro ao fazer upload da foto: ' + (error.detail || 'erro desconhecido'));
}

const uploadData = await uploadRes.json();
fotoPerfil = uploadData.url;
}

const payload = {
  nome: formData.nome,
  email: formData.email,
  telefone: formData.telefone,
  endereco: formData.endereco
};

if (fotoPerfil && !fotoPerfil.startsWith('data:')) {
  payload.foto_perfil = fotoPerfil;
}

const res = await fetch(`${API_URL}/api/v1/usuarios/me`, {
  method: 'PATCH',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`
  },
  body: JSON.stringify(payload)
});

if (!res.ok) {
  const error = await res.json();
  throw new Error(error.detail || 'Erro ao atualizar perfil');
}
} finally {
  setLoading(false);
}
};

---
```

4. Observacoes

- Trechos extraídos do projeto real, apenas reduzidos para apresentação.
- O código completo permanece sob controle do titular.
- Este documento pode ser anexado no pedido de RPC como material técnico.